

# COMP101

## Introduction to Programming

### 2025-26

## Lecture-14

### Selection Nested

# Nested if - colour coded

```
#candidates for election must be female minimum age 21
candidate = input ("Candidate gender: ") female
age = int(input("Enter age of candidate: ")) 25
```

```
if candidate == "female":
    if age >= 21:
        print ("female 21 or over")
        print ("accept")
    else:
        print ("female, but not 21")
        print ("dismiss")
else:
    print ("Not female")

print ("construct exited")
```

**if** executes when 'female' is input (i.e. **if** = True)  
Execute indented code under the **if** and ignore **else**:  
Indented code is a 'nested if with else' ('inner if with else')

**if** executes when 21 or more is input (i.e. **if** = True)  
Execute the indented code under the **if** and ignore **else**:

i.e. **if** executes when: **if** = True and **if** = True

**else** executes when input isn't 'female' (i.e. **if** = False)  
Don't execute the inner **if else**

**else** executes when **if** = 'female' but **if** is less than 21  
i.e. **else** executes when **if** = True and **if** = False

# Nested if compound statement

```
candidate = input ("Enter gender of candidate: ") female
age = int(input("Enter age of candidate: ")) 25
```

#compound expression

```
if candidate == 'female' and age >= 21:
    print ("female 21 or over - accept")
```

if executes only when both tests = True

```
elif age < 21:
    print ("Underage - not yet 21 - dismiss")
```

elif executes only when one or both if tests = False.  
We don't know which one is False so this elif tests for age, leaving gender test for the else.  
We could do this the other way round.

```
else:
    print ("Candidate not female - dismiss")
```

else 'captures the exception':  
It executes when if = F and elif = F

```
print ("construct exited")
```

# Nested if compound statement

```
candidate = input ("Enter gender of candidate: ") female
age = int(input("Enter age of candidate: ")) 25
```

#compound expression

```
if candidate == 'female' and age >= 21:
    print ("female 21 or over - accept")
```

if executes only when both tests = True

```
elif candidate == 'female' and age < 21:
    print ("Female but underage - dismiss")
```

#elif expanded to a compound expression using 'and':

In the if statement, one or both tests are false – we don't know which. This elif tells user which test was false in the if-statement

```
else:
    print ("Candidate not female - dismiss")
```

```
print ("construct exited")
```

else 'captures the exception': executes when if = F and elif = F

# Nested if compound statement

```
candidate = input ("Enter gender of candidate: ") female
age = int(input("Enter age of candidate: ")) 25
```

```
#compound expression
if candidate == 'female' and age >= 21:
    print ("female 21 or over - accept")

elif candidate == 'female' and age < 21:
    print ("Female but underage - dismiss")

else:
    print ("Candidate not female - dismiss")

print ("construct exited")
```

Test considerations: There are 3 conditionals – each evaluates T or F  
We employ (3 \* 2 = 6) tests as minimum:

‘age’: has range=21: test the ‘range boundaries’  
range=21, range=range+1 and range=range-1 in case ‘off-by-one’

‘candidate’: has specified input: test candidate = T and candidate = F

| Test# | i/p (gender,age) | Expected | Actual  |
|-------|------------------|----------|---------|
| 1     | female,21        | accept   | accept  |
| 2     | female,22        | accept   | accept  |
| 3     | female,20        | dismiss  | dismiss |
| 4     | male,21          | dismiss  | dismiss |
| 5     | male,22          | dismiss  | dismiss |
| 6     | male,20          | dismiss  | dismiss |

# Nested if using elif

```
score = int(input("Enter score: "))
```

```
if score >= 90:
```

```
    print ("A")
```

```
else:
```

```
    if score >= 80:
```

```
        print ("B")
```

```
    else:
```

```
        if score >= 70:
```

```
            print ("C")
```

```
        else:
```

```
            if score >= 60:
```

```
                print ("D")
```

```
            else:
```

```
                print ("F")
```

An 'okay' structure:  
Every if has an else:

We have four if and four  
else – it 'balances'

Can improve this with  
better logic

#this is a better structure

```
score = int(input("Enter score: "))
```

```
if score >= 90:
```

```
    print ("A")
```

```
elif score >= 80:
```

```
    print ("B")
```

```
elif score >= 70:
```

```
    print ("C")
```

```
elif score >= 60:
```

```
    print ("D")
```

```
else:
```

```
    print ("Fail")
```

We associate one else  
with one if:

A good structure has  
one 'if' at the start, one  
'else' at the end (to  
catch the exception);

The nested if is created  
using as many 'elif' as  
needed, each with its  
own test

# Nested if

```
colour = input ("Enter a colour from the spectrum: ") Indigo
```

```
if len(colour) <=6:
```

```
    print ("Colour is in spectrum")
```

```
        if len(colour) == 3:
```

```
            print ("Colour is red")
```

```
        elif len(colour)== 4:
```

```
            print ("Colour is blue")
```

```
        elif len(colour) == 5:
```

```
            print ("Colour is green")
```

```
        else:
```

```
            print ("Colour is any of Orange - Yellow - Indigo - Violet")
```

```
else:
```

```
    print ("Colour is not in spectrum")
```

Alternative nested if:

The outer if has one else (good) and the inner if ('nested if') has one if at the start, one else at the end and as many elif as needed in between (also good)

Not an elegant solution – look at all the 6-letter colours not specifically specified – confusing for user. Choose better tests

# if – elif – else

- A 1-year-old dog is equivalent to a 14-year-old human;
- A 2-year-old dog is equivalent to a 22-year-old human;
- Every year over 2-years-old is equivalent to 5 years human
- Find the human age of the dog (needs three tests: age is 1, 2, or >2)

| If – elif (no else)                                                                                                                                                                                                            | else – for exception handling                                                                                                                                                                                                                                                                                                      |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre>dog_Age = int(input ("Dog\'s age?: ")) if dog_Age == 1:     print ("14 years human")  elif dog_Age == 2:     print ("22 years human")  elif dog_Age &gt;2:     human_Age = (22+(dog_Age-2)*5)     print (human_Age)</pre> | <pre>#maximum dog age = 20 years old dog_Age = int(input("Dog\'s age?: ")) if dog_Age == 1:     print ("14 years human")  elif dog_Age == 2:     print ("22 years human")  elif dog_Age &gt;2 and dog_Age &lt;=20:     human_Age = (22+(dog_Age-2)*5)     print (human_Age)  else: #exception handling     print ("Too old")</pre> |

# if-elif-else: first draft

```
# United Kingdom = England, Ireland, Scotland, Wales
bornIn = input("Where in the UK where you born?: ")
if bornIn == "England":
    print("You are English")
    print("Capital is London")
elif bornIn == "Ireland":
    print("You are Irish")
    print("Capital is Belfast")
elif bornIn == ("Scotland"):
    print("You are Scottish")
    print("Capital is Edinburgh")
else:
    print("You are Welsh")
    print("Capital is Cardiff")
```

1. If statement executes when 'England' is input  
If statement is ignored when 'england' is input

2. Execution passes to the next un-indented line (elif). The test fails again.

3. Execution passes to the next un-indented line (elif). The test fails again.

4. All tests have failed; else executes for the exception:  
Everyone is Welsh and capital is Cardiff

# if- elif – else: Second draft

```
bornIn = input ("Where in the UK where you born?: ")
```

```
if bornIn == ("England") or bornIn == ("england"):
```

```
    print ("You are English")
```

```
    print ("Capital is London")
```

```
elif bornIn == ("Ireland") or bornIn == ("ireland"):
```

```
    print ("You are Irish")
```

```
    print ("Capital is Belfast")
```

```
elif bornIn == ("Scotland") or bornIn == ("scotland"):
```

```
    print ("You are Scottish")
```

```
    print ("Capital is Edinburgh")
```

```
else:
```

```
    print ("You are Welsh")
```

```
    print ("Capital is Cardiff")
```

Boolean operator (or) used to test for 'England' or 'england'

# if – elif – else:

## Final draft – recommended approach

```
bornIn = input ("Where in the UK where you born?: ")
bornIn = bornIn.upper()                #use a function to change the case to upper or lower

if bornIn == ("ENGLAND") :
    print ("You are English")
    print ("Capital is London")

elif bornIn == ("IRELAND"):
    print ("You are Irish")
    print ("Capital is Belfast")

elif bornIn == ("SCOTLAND"):
    print ("You are Scottish")
    print ("Capital is Edinburgh")

else:
    print ("You are Welsh")
    print ("Capital is Cardiff")
```

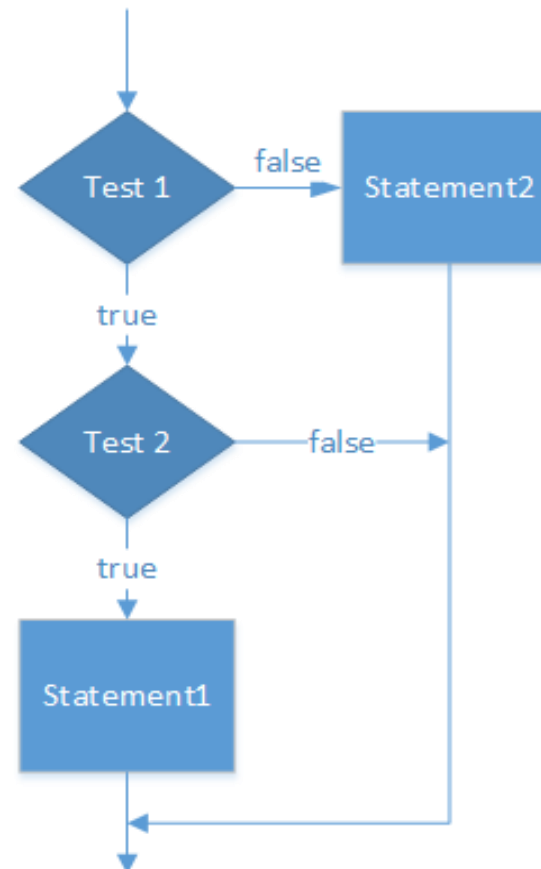
# Dangling Else problem (logic error)

<https://icarus.cs.weber.edu/~dab/cs1410/textbook/3.Control/if.html>

- Caused by over-dependence of indentation

Pseudocode:

```
if (test1)
    if (test2)
        statement1
else
    statement2
```



The *indentation* implies:

Execute statement 1 only if both Test 1 and Test 2 are true;

Execute statement 2 if test 1 is false

i.e. programmer wants else clause to match the first if;

**BUT ...**

**The else always matches the 'nearest' if, i.e. the first one above it (which is test2)**

The logic is at fault

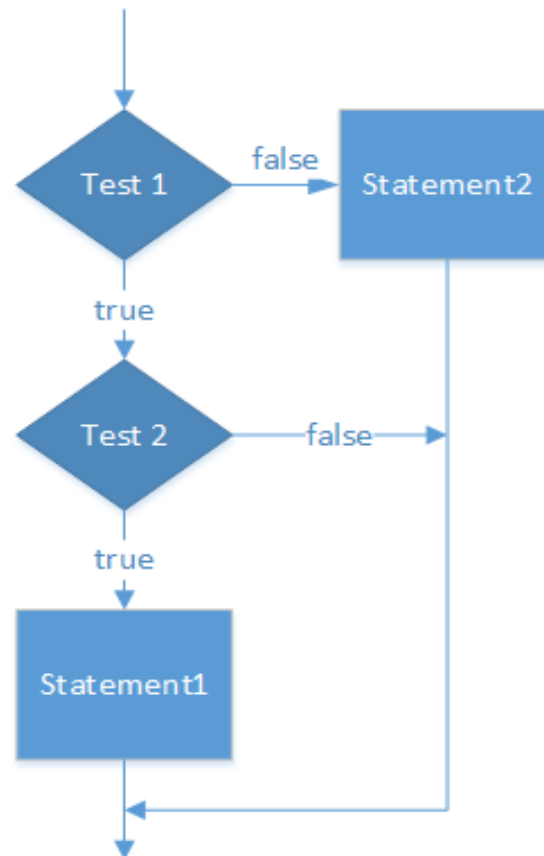
# Dangling Else problem (logic error)

<https://icarus.cs.weber.edu/~dab/cs1410/textbook/3.Control/if.html>

- Caused by over-dependence of indentation

Pseudocode:

```
if (test1)
  if (test2)
    statement1
  else
    statement2
else
  print message
```



The *indentation* implies

Execute statement 1 only if both Test 1 and Test 2 are true;

Execute statement 2 only when Test 1 or Test 2 is false;

Ideally put another else at the end (to 'line up' with the first if).

Now every if has an else that associates with it.

This else at the end may not do much but can tell the user the selection has ended – may help debugging.

Note on testing:

There are 2 tests – each test has 2 outcomes (T/F)

Need a minimum of 4 tests (2\*2)

```

If A:
    B
    D
    if D:
        E
        if E:
            G
        else:
            F
            if F:
                G
            else:
                C
    else:
        F
        if F:
            G
        else:
            C
else:
    C
    F
    if F:
        G
    else:
        C

```

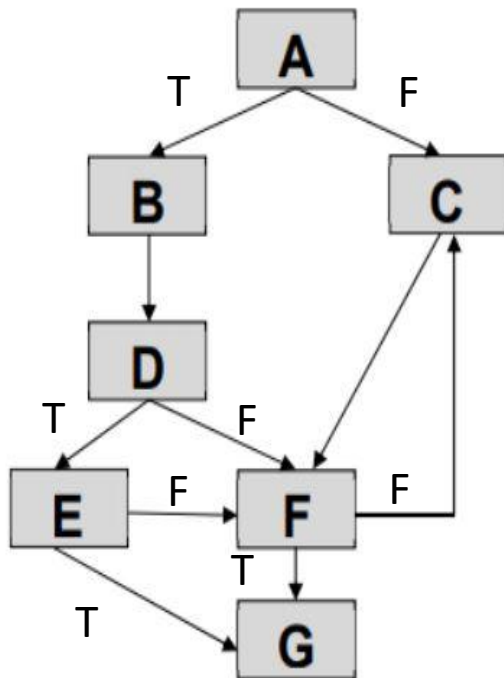
Complicated, hard to understand

Analysis:

- four conditional statements (if): A, D, E, F
  - These need testing
- Three non-conditionals: B, C, G
  - These do not have any test requirements
  - They execute as part of if being T or F
- Six if-statements, six else statements = balance
- ?When does B or C or G execute

Flowchart Testing – see tutorials

# Flowchart testing



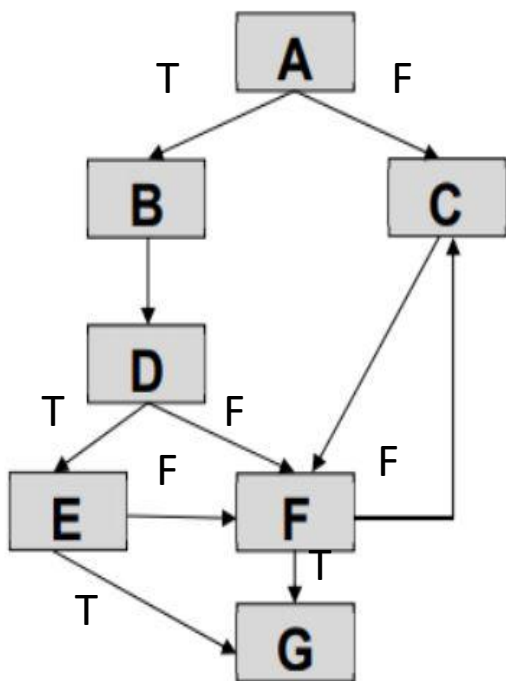
In the diagram, there are four conditions/decisions: A, D, E, F  
A,D,E,F are 'conditionals'  
An important test goal is 100% decision coverage.  
Employ tests for all the paths through the chart

## Test the conditionals

Test to cover condition A  
Test to cover condition D  
Test to cover condition E  
Test to cover condition F

Flowchart Testing – see tutorials

# Flowchart Testing



There are four conditions/decisions (‘conditionals’): A, D, E, F  
There are three non-conditionals: B,C,G

| Scenario | i/p       | Expected |
|----------|-----------|----------|
| 1        | A = True  | B, D     |
| 2        | D = True  | E        |
| 3        | E = True  | G        |
| 4        | F = True  | G        |
| 5        | A = False | C, F     |
| 6        | D = False | F        |
| 7        | E = False | F        |
| 8        | F = False | C        |

#B,D execute in sequence when A=T

#C,F execute in sequence when A=F

#C,F iterate depending on loop condition in C

This is not a test table but simply a table to help us see what should execute and when.  
We are using it here to check the flow of the logic and that we have covered all possible paths.  
We could do some simple ‘stub coding’ to see that if A=T then B and D execute etc.  
The code would not do much but programmer could code themselves a message to show the path is working